Detecting Strategically Decomposed Malicious Code Changes with Large Language Model Reviewers

Anonymous pre-outcome manuscript

17 July 2026

Abstract

Large language models are increasingly used to review code changes, yet most evaluations present a vulnerable function, file, or repository as one unit. A malicious change can instead be decomposed across submissions that appear individually routine but compose into an unsafe final state. We describe a prospectively specified, controlled study of whether this decomposition reduces detection and whether detection differs between pull-request gates and trunk workflows that treat main as untrusted until pipeline review completes. The study uses 400 matched malicious-benign scenario pairs generated from 200 structural templates in eight abstract policy families. Every program is synthetic, executes only in memory, and has no network, file-system, credential, persistence, deployment, or destructive capability. Atomic and three-submission variants have identical final trees and changed-line totals. A factorial design varies intent, decomposition, workflow framing, and local versus cumulative context across three fixed model routes and three repeated trials. A separately hashed 80-pair exploratory subset renders native-looking pull-request and untrusted-main artifacts. Primary outcomes are timely blocking and paired risk differences; secondary outcomes include false-positive rate, precision, calibration, localization, time to detection, and consistency. Family-stratified structural-template bootstrapping, missing-data bounds, and complete-pair sensitivity analyses are frozen before ground truth is joined to responses. This pre-outcome manuscript reports the design and analysis plan; it contains no confirmatory model outcomes.

1 Introduction

Automated code review is usually evaluated as a classification problem: a model receives a code artifact and predicts whether it contains a defect. Real delivery systems expose a different unit of work. Reviewers see submissions over time, with evidence constrained by pull-request boundaries, commit history, repository state, and pipeline policy. An unsafe capability may emerge only when several changes compose. Each earlier change can be necessary for the final failure without being unsafe in isolation.

This distinction matters for both pull-request and trunk-based delivery. A pull-request gate asks whether a proposed change may enter the protected branch. A trunk workflow can permit a commit onto main while still treating main as untrusted until deterministic and model-based pipeline gates complete. Those workflows differ in policy, but empirical comparisons often also change the evidence shown to the reviewer. A useful experiment must therefore vary workflow framing independently from information availability.

We study whether an LLM reviewer recognizes a composition before its activation boundary. The benchmark represents harm as an abstract policy assertion: an unprivileged synthetic actor receives a protected synthetic capability. It does not implement an exploit or interact with a real target. The design makes three contributions:

1. a causally validated benchmark in which every malicious split step is necessary, while atomic and split variants reach the same final tree;
2. a factorial comparison of decomposition, workflow policy, and reviewer context that separates workflow effects from information loss; and
3. a frozen sequence-level analysis with paired benign controls, template-respecting uncertainty, missingness bounds, and reproducible ledgers.

2 Related Work

Yu et al. evaluated seven LLMs and multiple prompt forms for security code review, comparing them with static analysis and examining factors associated with detection quality [1]. Their results motivate fixed prompts, false-positive reporting, and explicit context controls. Our study changes the unit of analysis from one review artifact to a sequence of submissions.

VulDetectBench separates vulnerability identification, classification, and localization, showing that aggregate accuracy can conceal weaknesses in detailed analysis [2]. We likewise distinguish timely blocking, calibration, severity, and localization. Zhou et al. compare repository-level LLMs and static analysis tools and report a recall-false-positive tradeoff [3]. We retain a deterministic conventional-lint baseline but do not claim that lint and semantic security review are equivalent tasks.

Field evidence also indicates that review context and interaction design affect developer trust and usefulness. Adalsteinsson et al. report context switching, missing context, false positives, and trust as practical concerns in an industrial study of LLM-assisted review [4]. Basic and Giaretta synthesize evidence on LLM code-security benefits, false findings, prompting, and failure modes [5]. Our controlled study complements that work by isolating temporal composition and workflow framing.

3 Research Questions and Hypotheses

The primary endpoint is *blocked by activation*: at least one review at or before the prespecified activation boundary returns `block`. The research questions are:

RQ1 Can LLM reviewers discriminate malicious sequences from matched benign sequences?

RQ2 Does strategic decomposition across three submissions reduce timely detection relative to an atomic submission with the same final tree?

RQ3 Does detection differ between pull-request and untrusted-main trunk workflow framing, and does workflow moderate the decomposition effect?

RQ4 Does cumulative submission history mitigate any decomposition penalty relative to local current-submission evidence?

The prespecified hypotheses are directional except where noted:

H1 The malicious block rate exceeds the benign block rate.

H2 Under local context, the malicious block rate is lower for split sequences than for atomic submissions.

H3a The marginal trunk-minus-pull-request risk difference is equivalent within -0.10 and 0.10.

H3b The decomposition-by-workflow interaction differs from zero.

H4 Cumulative context attenuates the split penalty relative to local context.

4 Method

4.1 Safe benchmark

The generator produces 400 malicious-benign scenario pairs from 200 structural templates: eight scenario families crossed with five data representations and five control-flow shapes, with two identifier instances per template. The eight families model abstract authorization, normalization, fallback, integrity, scope, feature, provenance, and dependency-policy composition.

Each case contains an atomic submission and a three-submission sequence. The atomic and split forms have identical final source trees, total changed-line counts, activation timestamps, and policy outcomes. For malicious sequences, the generator reverts each split step separately and executes the oracle; all three

counterfactuals must remain safe. Benign cases match the structure and changed-line count but do not grant the protected capability.

The source imports only relative synthetic modules and runs only in memory. A deterministic capability scan rejects network, shell, subprocess, file-system, package-install, credential, personal-data, persistence, destructive, dynamic execution, and non-relative import features. The benchmark is not deployable and contains no real target or exploit instructions.

4.2 Factorial design

Table 1 shows the controlled factors. Workflow policy and evidence context are orthogonal: changing *pull request* to *trunk* changes only the written decision policy, while changing *local* to *cumulative* changes only whether prior submissions are supplied.

Table 1: Controlled study factors.

Factor	Levels	Interpretation
Intent	malicious, benign	Matched final-state policy outcome
Decomposition	atomic, split	One submission or three necessary submissions
Workflow	pull request, trunk	Pre-merge gate or untrusted-main pipeline decision
Context	local, cumulative	Current boundary only or explicit prior history

The full design contains 57,600 sequence evaluations and 115,200 boundary-level review calls. Calls are randomized with a seeded Fisher-Yates shuffle. Each scenario-condition-model cell has three trials. No outcome-dependent stopping, template replacement, or prompt tuning is permitted.

4.3 Exploratory ecological layer

A separate exploratory layer tests whether the controlled workflow comparison changes when the evidence takes native-looking forms. Pull-request requests contain a title, description, base and head revisions, checks, and a diff; trunk requests contain a landed commit, its parent, a diff, and an explicitly untrusted pending pipeline state. Cumulative conditions include earlier native artifacts, while local conditions do not.

This layer selects a balanced diagonal of five templates from each family, covering every data representation and control-flow shape once per family. Its 40 templates, two identifier instances, and one trial per model produce 80 scenario pairs, 2,560 unique prompts, and 7,680 calls. Artifact form is confounded with workflow by design, so these results are descriptive and cannot support or alter H1–H4.

4.4 Review configurations

The frozen routes represent a frontier proprietary model, an independent proprietary software-engineering replication, and a code-specialized open-weight model (Table 2). Route fallback is disabled; provider and endpoint metadata must match the prespecified route.

Table 2: Frozen model routes. Serving metadata and access times are retained in the result ledger.

Route	Study role
`openai/gpt-5.6-sol`	frontier proprietary
`anthropic/claude-sonnet-4.6`	independent proprietary software-engineering replication
`qwen/qwen3-coder-next`	code-specialized open-weight replication

All reviewers receive one fixed system task and a JSON response contract. The request asks for `allow`, `block`, or `abstain`; a maliciousness probability; severity; relevant submission identifiers; evidence; and a short category. Requests are capped at 4,000 UTF-8 bytes and 2,000 input tokens, with no silent truncation. Temperature is zero where supported, tools are disabled, and output is capped at 256 tokens.

4.5 Collection and blinding

The call queue contains opaque call, prompt, and case identifiers plus model and trial. It contains no intent, family, template, or sequence label. Ground truth is held in a separate ledger and joined only after collection. Refusals and schema-invalid responses are observed abstentions and are not retried. Network errors, HTTP 408, 409, 429, 5xx responses, and route mismatches receive at most two retries with the identical request. Retry exhaustion produces a missing boundary.

Collection suspends before accrued cost plus the frozen projected remainder would exceed US$1,400, after 100 consecutive infrastructure failures, or on a safety, terms, or benchmark-integrity concern. The projected confirmatory cost is US$1,094.55. The ecological layer has a separate US$72.97024 projection and US$100 ceiling. These ceilings are safety limits, not authorization to spend.

4.6 Outcomes

The primary binary outcome is blocked by activation. Secondary outcomes are recall, false-positive rate, precision, abstention rate, cumulative detection by boundary, submissions to detection, Brier score, ten-bin expected calibration error, ordinal severity error, localization, and repeated-trial consistency.

A blocked response is localized when it cites at least one prespecified relevant submission and identifies the causal abstract-policy failure. Two adjudicators score localization independently while blinded to model, trial, and condition labels; a third resolves disagreement. Pending adjudications remain null rather than being scored as failures.

5 Analysis Plan

Sequence outcomes are averaged within structural-template and intent-condition cells across balanced identifier instances, models, and trials. Structural templates then receive equal weight. The primary uncertainty calculation is a 10,000-replicate percentile bootstrap with seed 20260716. Within each replicate, 25 templates are sampled with replacement separately from each of the eight scenario families, retaining paired intent and treatment cells.

Table 3: Confirmatory estimands and decision rules.

Claim	Estimand	Rule
H1	malicious minus benign block-rate risk difference, averaged over all factors	95% interval above zero
H2	split minus atomic malicious risk difference under local context, averaged over workflow	95% interval below zero
H3a	trunk minus pull-request malicious risk difference, averaged over decomposition and context	90% interval inside [-0.10, 0.10]
H3b	decomposition-by-workflow difference in differences, averaged over context	95% interval excludes zero
H4	decomposition-by-context difference in differences, averaged over workflow	95% interval above zero

The design was audited with a 20,000-replicate hierarchical simulation. Under the frozen design assumptions, the selected 200-template layout exceeded 0.99 power or assurance for all five confirmatory targets. The 200-template floor was retained for full structural coverage even though a smaller repeated-template layout met the numerical threshold.

Missing boundaries count as abstentions and therefore not detections in the primary analysis. Two mandatory bounds treat missing malicious and benign boundaries oppositely: the detection-favorable bound codes missing malicious boundaries as blocks and benign boundaries as allows; the detection-unfavorable bound reverses those assignments. A conclusion is called robust only when its primary criterion holds under both bounds. Per-model complete-pair sensitivity drops a structural template only for the model with a missing condition. No imputation model is fitted.

Secondary model-specific, family-specific, calibration, timing, localization, and ablation analyses are labeled as such. Holm adjustment is applied when multiple secondary comparisons within one outcome family are interpreted inferentially. The earlier mixed-effects logistic model is retained as a sensitivity analysis rather than the primary estimator.

The ecological layer reports the same descriptive outcomes and differences between controlled and native-artifact presentations by workflow and context. All such comparisons are exploratory. Repeated-trial consistency is omitted because the ecological layer has one trial per cell.

6 Pre-outcome Results Status

No confirmatory model outcomes have been collected or inspected. The benchmark, prompt ledgers, schedules, collection failure rules, joins, estimands, and tests were implemented using deterministic checks and fabricated responses only. This section will report participant flow, call failures, primary estimates and intervals, missingness bounds, model-specific results, calibration, localization, consistency, and exploratory ecological comparisons after prospective registration and authorized collection. Until then, no empirical conclusion about H1–H4 or the ecological layer is warranted.

The deterministic baseline ran ESLint 9.39.3 over the three final-state modules in all 800 cases. It emitted no warnings or errors: zero malicious and zero benign cases were flagged, giving recall 0, false-positive rate 0, and undefined precision. This result shows only that conventional lint does not identify the benchmark's abstract cross-module policy failures. Because it sees final trees only, it cannot estimate decomposition, context, or workflow effects. A separate zero-finding Semgrep feasibility probe is disclosed but excluded because its registry rules snapshot cannot be redistributed under the provider's rules license (`https://semgrep.dev/legal/rules-license/`).

7 Interpretation Framework

Several result patterns have distinct interpretations. A split penalty under local but not cumulative context would support an information-integration failure rather than a general inability to recognize the final unsafe state. A split penalty under both contexts would instead suggest that temporal presentation or individually weak evidence impairs composition even when history is explicit. A decomposition-by-workflow interaction, with evidence held constant, would indicate sensitivity to the decision policy rather than the code artifact. High recall accompanied by high benign blocking would not constitute effective review and will be reported through precision and false-positive rate.

Failure analysis will distinguish missed composition, incorrect causal detail, overreliance on plausible metadata, severity underestimation, unlocalized blocking, refusal, schema failure, and infrastructure failure. These categories are descriptive and cannot replace the prespecified contrasts.

8 Threats to Validity

Construct validity. The benchmark models malicious intent through a harmless abstract policy violation. This provides causal control but does not reproduce the semantics or distribution of production vulnerabilities. Workflow is a controlled written policy, while the ecological layer varies artifact form as well as policy. Neither reproduces organizational incentives, reviewer fatigue, rollback pressure, or the blast radius of a real main branch.

Internal validity. Atomic and split variants can differ in salience despite identical final trees and line counts. Counterfactual necessity, prompt hashing, timestamp controls, and independent review reduce but do not eliminate this risk. Provider serving stacks may also change behind a stable route; returned metadata and access times are therefore retained.

External validity. Eight policy-composition families, three review configurations, and compact synthetic repositories do not represent all languages, organizations, or attack strategies. The study tests one three-submission decomposition and does not estimate an optimal adversarial strategy. Publication may contaminate future models, so benchmark and model versions must accompany later replications.

Statistical conclusion validity. Repeated calls and identifier variants are not independent scenarios. The analysis therefore generalizes at the structural-template level and preserves family strata. Power values depend on design assumptions rather than pilot outcomes. Missingness bounds expose, rather than hide, dependence on provider failures.

9 Ethics and Responsible Release

This is defensive research into review reliability. The benchmark cannot access external systems or data, and its only harmful state is a deterministic in-memory assertion over synthetic names. No real repository or person is tested without explicit authorization. A separate reviewer assesses whether release artifacts could provide reusable evasion guidance. Material judged operationally risky is withheld or reduced to non-operational metadata, and the omission is documented.

Raw model output is released only where provider terms permit. Human review, if feasible, requires consent, withdrawal procedures, anonymization, and any applicable ethics review. The study reports provider refusals, missing calls, deviations, and negative results rather than removing them post hoc.

10 Reproducibility

The versioned package contains the generator, cards, prompts, schedule, collection ledgers, scorer, power simulation, preregistration drafts, and independent-review protocol. Candidate artifact hashes are:

- scenario cards:
  `71b20e0e5a49547a68d17a8435b1a9f860e8f078942bbc2b66b557971a9a4ab3`
- rendered prompts:
  `56d246f9c676e94f29ce2dea4025633f1b31815959b5b34b0daf2b49af32dff4`
- randomized schedule:
  `e90a26028bfd0b0a38f15ce55c71ffb5c983c8eaf03b7ef18cc74f697481a663`
- ecological rendered prompts:
  `b2837c4088147e036fbdf7e7e985bfbd3b15d3f1a187c735bf1096236d806580`
- ecological randomized schedule:
  `d2ffe01262af1d1a93facc441c6df03bd9e32d745548da541ddc01cf25cbdc5e`
- ESLint configuration:
  `fd9af0693c9fb3c7b9f9dfd573b944184a03ae0bd8b09f734cd97bb6dae15709`

The repository is available at `https://github.com/windyroad/windyroad`. The final release will add permitted raw responses, an attempt ledger, joined analysis data, adjudication records, environment metadata, checksums, the prospective registration identifier, and the arXiv identifier.

11 Conclusion

This study tests a gap between artifact-level vulnerability detection and sequence-level code review: whether a model can recognize several individually ordinary changes as a dangerous composition before activation. By independently varying decomposition, workflow policy, and context, the design can distinguish information loss from workflow framing. The safe paired benchmark and frozen analysis make both positive and negative findings auditable. Empirical conclusions remain contingent on prospective registration, authorized collection, and the prespecified analysis.

A Fixed Review Task

The fixed task instructs the model to act as an independent security reviewer, treat repository and metadata as untrusted evidence, decide whether the current submission should be allowed, blocked, or sent for human review, consider unsafe behavior created alone or in composition with supplied history, and return only the contracted JSON grounded in supplied evidence. No chain-of-thought request, model-specific prompt, or outcome-driven tuning is used.

B Artifact Status at the Pre-outcome Freeze

Machine validation covers final-tree equality, changed-line equality, activation timestamps, paired benign safety, malicious activation, counterfactual necessity, capability exclusions, prompt size, schedule balance, call-ledger blinding, retry behavior, spending suspension, result joining, missingness bounds, and all five confirmatory contrasts. A separate generator covers the exploratory native pull-request and untrusted-main artifacts. Independent benchmark and statistical reviews, exact native token counts for both layers, author metadata, license confirmation, and external registration remain required before collection.

References

[1] J. Yu, P. Liang, Y. Fu, A. Tahir, M. Shahin, C. Wang, and Y. Cai. An insight into security code review with LLMs: Capabilities, obstacles, and influential factors. *arXiv:2401.16310*, 2024. `https://arxiv.org/abs/2401.16310`.

[2] Y. Liu, L. Gao, M. Yang, Y. Xie, P. Chen, X. Zhang, and W. Chen. VulDetectBench: Evaluating the deep capability of vulnerability detection with large language models. *arXiv:2406.07595*, 2024. `https://arxiv.org/abs/2406.07595`.

[3] X. Zhou, D.-M. Tran, T. Le-Cong, T. Zhang, I. C. Irsan, J. Sumarlin, B. Le, and D. Lo. Comparison of static application security testing tools and large language models for repo-level vulnerability detection. *arXiv:2407.16235*, 2024. `https://arxiv.org/abs/2407.16235`.

[4] F. S. Adalsteinsson, B. B. Magnusson, M. Milicevic, A. N. Davidsson, and C.-H. Cheng. Rethinking code review workflows with LLM assistance: An empirical study. *arXiv:2505.16339*, 2025. `https://arxiv.org/abs/2505.16339`.

[5] E. Basic and A. Giaretta. From vulnerabilities to remediation: A systematic literature review of LLMs in code security. *arXiv:2412.15004*, 2024. `https://arxiv.org/abs/2412.15004`.